COMP 3105 – Assignment 4 Report
– Fall 2025 –

Due: Sunday November 30, 2025 23:59.

Group 51
Andrew Wallace - 101210291
Christer Henrysson - 101260693

Getting started

Note that Python 3.11 is used for this assignment. Please install requirements using virtual environment via:

```
python3.11 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Question 1 (5%) Learning and Classifying

Please see `A4codes.py` for the implementation of `learn(path_to_in_domain, path_to_out_domain)`.

Please see `A4codes.py` for the implementation of `compute_accuracy(path_to_eval_folder, model)`.

Question 2 (5%) Explanation

Overview

In traditional machine learning, it is assumed that the training and test data come from the same distribution. However, in this problem the training and test data come from different distributions (images from an "in-domain" set and images from an "out-domain"). Furthermore, we are presented with labelled data for the in-domain set but have unlabelled data for the out-domain set at training time. Our objective is to train a model that learns to accurately classify in-domain images while simultaneously inaccurately classifying out-domain images at evaluation time.

Initially, we viewed this problem as a reverse domain adaptation problem. In domain adaptation, the objective is to train a model from some source data that generalizes to some target domain through minimizing the difference between source and target distributions [1]. In our problem, we aim to do the opposite - train a model that does not generalize to another distribution.

Domain-Adversarial Training of Neural Networks

One approach to domain adaptation is Domain-Adversarial Training of Neural Networks (DANN). DANN focuses on learning features that combine discriminativeness and domain invariance through the optimization of underlying features, a label predictor (classifier), and a domain classifier [2]. The Domain-Adversarial Neural Network can be broken down into three parts.

1. Deep Feature Extractor
   The Feature Extractor is a deep neural network responsible for extracting features from the input.

2. Label Predictor (Classifier)
   The Label Predictor is typically a deep multi-layer perceptron (MLP) that is responsible for predicting class labels.

3. Domain Classifier
   The Domain Classifier is typically a shallow MLP that predicts whether a feature (from

the feature extractor) comes from source or target domain.

This gives two different classifications and loss functions, one for label classification (L_y) and one for domain classification (L_d).The addition of the domain classifier allows you to add a "gradient reversal layer" that multiplies the gradient by a certain negative constant (computed from the loss L_d) during backpropagation, encouraging the model to be domain invariant [2]. This DANN model performs very well for domain adaptation tasks, however, our problem requires us to train a discriminative model that does the opposite.

DANN Inspired Implementation

Using DANN as inspiration, we initially setup our model in a similar way. We used a 6-layer convolutional neural network with ReLU activation as our feature extractor. This extracts core image features such as object parts and textures. We then had a 2-layer fully connected MLP with ReLU activation as our label classifier. Finally, we used a 2-layer fully connected MLP with ReLU activation as our domain classifier.

At training time, we are given labelled in-domain images and unlabeled out domain images. We preprocess these images and add domain classification labels (0 for in-domain and 1 for out-domain). These labels were later used in the calculation of domain loss.[1em] We then trained the model using both in-domain and out-domain image batches. The label classifier was used to predict class labels for the given feature. It's loss was calculated and is included in the total loss for backpropagation. For the domain classifier, instead of having a gradient reversal layer that encourages domain invariance, we simply added the domain loss to the total loss to encourage the model to produce features that allow the domain classifier to distinguish between in and out domains. We believed that the domain loss would act as a regularizer that would constrain the feature extractor to extract features that are meaningful not only to the class but also to the domain. During evaluation the model then would've (ideally) extracted features from the image that are relevant to both class and domain. However, after implementing this architecture, we did not get the expected results. The table below shows the results of the algorithm.

Learning Rate	Domain Step Size λ	In-Domain Accuracy (%)	Out-Domain Accuracy (%)
0.001	0.8	73	57
0.0005	0.8	78	58
0.0001	0.8	74	59
0.001	0.1	70	54
0.0005	0.1	75	57
0.0001	0.1	71	60

Table 1: In-Domain and Out-Domain Test Accuracies with different hyper-parameters

In the table above, we decent training accuracy for the in-domain test set, however, we also see a high accuracy for the Out-Domain test set. Our goal with this model was to push

the Out-Domain accuracy to a "best guess" accuracy. With 10 classes, we'd expect to see an accuracy for Out-Domain closer to 10%. The results did not reflect this.

One possible reason for this is that our model generalized too much and under-fitted the in-domain dataset, allowing for higher accuracy for the Out-Domain. Based on the results, we decided to pivot to a new model.

New Model Design

In the new model design we wanted to keep the domain loss mechanism. In our previous model, we calculated the loss by how well the model predicted whether the features belonged to the In or Out domain. This approach not sufficiently bring the Out-Domain accuracy down toward 10%. So, our solution was to compare the probability distribution (logits) of the Out-Domain classification directly with a uniform distribution and calculate it's difference as the loss. With this approach we expected the model to keep the same accuracy for in-domain, but the Out-Domain accuracy to drop. While we did observe the latter, the in-domain accuracy dropped significantly (on average 40%).

Final Design

After further investigation, we found that the model was severely over-fitting the training data and was not generalizing enough for the in-domain dataset. For example, from one training run we we're seeing 100% accuracy for training data and 40-50% accuracy for test data. To fix this we implemented 3 main features:

1. Reduced the complexity of the CNN to 4 convolutional layers and 2 dense layers
2. Added batch normalization to the convolutional layers
3. Applied data augmentation to the training dataset
4. K-Fold cross validation with early stopping

We reduced the complexity of the CNN so that the model doesn't learn overly complex features in the training set. The batch normalization was added to stabilize training. Applying data augmentation allows the model to "see" more data without explicitly adding new images to the training set. This helps generalize and prevent over-fitting. Lastly, we applied K-Fold cross validation with early stopping as our main regularizer which monitors the validation loss, stopping training when it no longer improves.
With these features implemented our model's performance increased.

Final Result

After training our final model design with the provided training dataset, we observed the following results.

Epochs	Folds (k)	In-Domain Accuracy (%)	Out-Domain Accuracy (%)
100	5	61	42
100	1	58	41

Table 2: In-Domain and Out-Domain Test Accuracies with different hyper-parameters for Final Model

This problem was challenging in that the model needs to perform well on an in-domain dataset and poorly on an out-domain dataset. We initially attempted to invert the DANN framework by encouraging domain discriminability rather than invariance. This method did achieve a good in-domain accuracy, but at the cost of increasing out-domain accuracy. We then explored an alternate approach to calculate the domain loss by comparing the logits to a uniform distribution. This method successfully reduced the out-domain accuracy but at the cost of in-domain accuracy. Through further analysis, we discovered that over-fitting was the culprit of the poor accuracy. We refined our model by adding k-fold cross validation, reducing model complexity, and applying data augmentation. This greatly reduced the over-fitting and we saw more balanced accuracies as a result.

While the final model did not achieve the 10% "best guess" goal, it demonstrated great progress in lowering the out-domain accuracy while maintaining a moderate in-domain accuracy.

Question 3 (5%) Evaluation

For this question, you don't need to submit/write an answer. We will apply your learning method to different training sets and evaluate the performance of the learned model on hold-out test datasets

References

[1] A. Farahani, S. Voghoei, K. Rasheed, and H. R. Arabnia, "A brief review of domain adaptation," A Brief Review of Domain Adaptation, https://arxiv.org/abs/2010.03978 (accessed Dec. 2, 2025).

[2] Y. Ganin et al., "Domain-Adversarial Training of Neural Networks," Journal of Machine Learning Research 17 (2016) 1-35, https://jmlr.org/papers/volume17/15-239/15-239.pdf (accessed Dec. 3, 2025).